# OpenAI SDK Tool Calls — Mechanism, Redaction, Firewall & Skill Loading

2026-05-06

## OpenAI SDK Tool Calls — Mechanism, Redaction, Firewall & Skill Loading

---

### 1. The Core Problem

LLMs can only generate text. They cannot browse the web, query a database, or check live data. **Tool calls** are the bridge between "text generation" and "real-world actions." The LLM never runs code itself — it *requests* that your code runs something on its behalf.

---

### 2. The Mental Model: A Request–Execute–Report Loop

```
Client code
|
+- defines tool schemas  ----------------------------------------+
+- sends [messages + tools] to API  -->  LLM reasons; picks tool |
|                                        |                        |
|  <-- tool_call response ---------------+                        |
|                                                                 |
+- executes real function(s)                                     |
+- appends tool result to messages                               |
+- sends back to API  -->  LLM sees result -> writes final answer |
                                                                  |
         ^                                                        |
         +-- loop repeats if LLM needs more tools ----------------+
```

Think of it as an assistant who can ask you to look things up:

```
You  → LLM: "What's the weather in NYC?"
LLM  → You: "Please call get_weather('NYC') and give me the result."
You  → (executes get_weather) → "72°F, sunny"
You  → LLM: "Tool result: 72°F, sunny"
LLM  → You: "It's 72°F and sunny in New York City!"
```

---

## 3. The Mechanism — Five Steps

### Step 1 — Define tools (JSON schema, not code)

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            },
            "required": ["city"]
        }
    }
}]
```

This is a *description* — the LLM reads it to know what capabilities exist. No code runs here.

### Step 2 — LLM responds with a `tool_calls` block (not text)

```
{
  "role": "assistant",
  "tool_calls": [{
    "id": "call_abc123",
    "type": "function",
    "function": {
      "name": "get_weather",
      "arguments": "{\"city\": \"NYC\"}"
    }
  }]
}
```

Instead of answering directly, the model emits a structured request. Arguments are a JSON string — typed, validated, safe to parse.

### Step 3 — Your code executes the real function

```
if response.choices[0].message.tool_calls:
    for tc in response.choices[0].message.tool_calls:
        result = get_weather(json.loads(tc.function.arguments)["city"])
```

### Step 4 — Return the result as a `tool` role message

```
messages.append({
    "role": "tool",
```

```
    "tool_call_id": "call_abc123",   # links result to request
    "content": "72°F, sunny"
})
```

**Step 5 — Call the API again; LLM produces the final answer**

The LLM now has the data it needed and writes a normal text response.

---

## 4. Key Design Choices

| Design choice | Reason |
|---|---|
| LLM outputs a schema request, not executable code | Sandboxing — the model never runs arbitrary code |
| Tool result sent as a new conversation turn | Keeps full history; model sees all context |
| `tool_call_id` links request to result | One response can request **multiple tools in parallel** |
| Arguments are a JSON string | Typed, validated input; prevents injection |

---

## 5. The Redaction Step

In production (SafeChain path), **every message is sanitized before it leaves the client** — before hitting the firewall, before the LLM sees it. This is the redaction step.

**What it does**

Located in `llm/firewall_stack.py → sanitize_message()` and applied by `llm/safechain_client.py → _redact_message()`:

```
_CASE_ID_RE = re.compile(r"CASE-\d+")
_DIGIT_RUN_RE = re.compile(r"\d{6,}")

def sanitize_message(message: str) -> str:
    masked = _CASE_ID_RE.sub("[CASE-ID]", message)
    return _DIGIT_RUN_RE.sub("***MASKED***", masked)
```

| Pattern | Masked as | Example |
|---|---|---|
| `CASE-123456` | `[CASE-ID]` | Case identifiers |
| Any 6+ digit run | `***MASKED***` | Account numbers, SSNs, phone numbers |

**Where it sits in the pipeline**

```
Agent / SDK
    |
```

```
    |  messages = [system, user, tool_result, ...]
    |
    v
_redact_message()          <- sanitize every outbound message
    |
    |  long digits -> ***MASKED***
    |  CASE-\d+   -> [CASE-ID]
    v
_combine_messages()        <- flatten multi-turn -> single string
    |                         (SafeChain: only one human message)
    |
    v
SafeChain Firewall         <- HTTP POST; firewall evaluates content
    |
    v
LLM                        <- sees only masked identifiers
```

### Why before flattening?

Redaction runs on each message individually *before* they are combined into one string. This ensures:

1. **No PII leaks through concatenation** — joining messages before masking could create new 6-digit sequences at join boundaries.
2. **Retry safety** — `_inject_guidance()` also re-redacts every message when it appends firewall guidance on a retry, so redaction is idempotent across attempts.

### The `redact_payload` helper (deep recursion)

For tool *results* (which can be Pydantic models, dicts, or lists), `firewall_stack.redact_payload()` walks the entire structure:

```python
def redact_payload(payload):
    if isinstance(payload, str):    return sanitize_message(payload)
    if isinstance(payload, dict):   return {k: redact_payload(v) ...}
    if isinstance(payload, list):   return [redact_payload(v) ...]
    if isinstance(payload, BaseModel):
        return Type.model_validate(redact_payload(payload.model_dump()))
    return payload
```

This means even nested tool outputs — e.g., a `SpendingReport` with embedded account numbers — are fully masked before the LLM reads them.

---

## 6. Firewall Handling

After redaction, every API call passes through a firewall layer. The firewall has two implementations that share the same retry loop logic, held in `FirewallStack` (`llm/firewall_stack.py`):

| Implementation | File | Used when |
|---|---|---|
| `FirewalledAsyncOpenAI` | `llm/firewall_client.py` | Dev / direct OpenAI path |
| `SafeChainChatCompletions` | `llm/safechain_client.py` | Prod / SafeChain path |

Both are drop-in replacements for `openai.AsyncOpenAI` — the SDK sees the same interface and never knows which path is active.

### Shared state: `FirewallStack`

```
class FirewallStack:
    def __init__(self, logger, max_retries=2, concurrency_cap=3):
        self.logger = logger
        self.max_retries = max_retries
        self.semaphore = asyncio.Semaphore(concurrency_cap)
```

`FirewallStack` owns three things:

- **logger** — emits `firewall_rejection` / `firewall_blocked` events
- **max_retries** — how many rejection-and-retry cycles are allowed (default 2)
- **semaphore** — caps concurrent in-flight requests (default 3) to stay within token-rate limits; every call acquires it before hitting the network

### The retry-with-guidance loop

Both implementations run the same loop (shown here for the OpenAI path in `firewall_client.py`):

```
messages = [_redact_message(m) for m in messages]    # 1. pre-redact
attempt = 0
while True:
    try:
        async with firewall.semaphore:               # 2. acquire slot
            return await base.create(messages=messages, ...)
    except FirewallRejection as e:
        firewall.logger.log("firewall_rejection", {..., "attempt": attempt})
        if attempt >= firewall.max_retries:
            firewall.logger.log("firewall_blocked", {...})
            raise                                    # 3a. give up
        attempt += 1
        messages = _inject_guidance(messages)        # 3b. inject hint, retry
```

On each rejection the loop does not simply retry with the same payload. It calls `_inject_guidance`, which:

1. Re-redacts every message (idempotent safety pass).
2. Appends `FIREWALL_GUIDANCE` to the first system message:

```
[IMPORTANT: Your previous response was blocked by the content firewall.
Avoid: raw account numbers, PII, role-injection patterns like [SYSTEM] or
[USER], code execution keywords (exec, eval, import). Use masked identifiers
and descriptive language instead of raw numeric values.]
```

The guidance tells the LLM *why* it was blocked and *what to change*, so the retry produces cleaner output rather than repeating the same violation.

### HTTP error mapping (SafeChain path)

The SafeChain HTTP layer raises generic exceptions. The shim translates them into typed errors before the retry loop sees them:

| HTTP status | Meaning | Action |
|---|---|---|
| 401 | Token expiry | Refresh the SafeChain model object; retry once immediately (outside the rejection loop) |
| 403 | Firewall block | `raise FirewallRejection("403", ...)` — enters the retry-with-guidance loop |
| 400 | Bad request | `raise FirewallRejection("400", ...)` — enters the retry-with-guidance loop |

The 401 path is special: it refreshes credentials and retries once *before* any guidance injection, because the cause is auth expiry, not content policy.

#### `FirewallRejection`

```python
class FirewallRejection(Exception):
    def __init__(self, code: str, message: str):
        self.code = code
        self.message = message
```

A typed exception that carries the error `code` (HTTP status or internal label) and a human-readable `message`. Callers that want to surface the rejection to the user can catch it by type; the retry loop catches it first and only re-raises after exhausting retries.

### Concurrency cap

The `asyncio.Semaphore(concurrency_cap=3)` is acquired around every network call. Without it, the orchestrator's parallel tool calls (report agent + N specialists fired simultaneously) could flood the upstream with a burst that exceeds the token-per-minute limit. Three concurrent requests is conservative; accounts on higher-tier plans can pass `concurrency_cap=N` when wiring `FirewallStack`.

### Flow summary

```
messages (redacted)
      |
      | async with semaphore      <- cap concurrent calls
```

```
    v
  API call
    |
    +-- success --> return ChatCompletion
    |
    +-- FirewallRejection
          |
          +-- attempts < max_retries?
          |         |
          |         yes --> _inject_guidance() --> retry
          |
          +-- no --> log firewall_blocked --> raise
```

---

## 7. Combined Picture (prod path)

```
1. User question arrives
2. Orchestrator composes messages + tool schemas
3.   REDACTION   every message sanitized (digits, case IDs masked)
4. Messages flattened into one string (SafeChain constraint)
5. SafeChain firewall evaluates; blocks or passes
6. LLM emits {"tool_call": {...}} JSON  (no native function-calling)
7. SafeChain shim parses JSON → synthesizes OpenAI tool_calls object
8. Agent SDK dispatches real tool (data query, etc.)
9. Tool result →   REDACTION   redact_payload() on the result struct
10. Result appended as "Tool result:" label in next flattened message
11. Steps 4-10 repeat until LLM emits {"output": {...}}
12. Final answer returned to user
```

---

## 8. How Markdown Skills Are Loaded

### What a skill file looks like

Every skill is a plain `.md` file with a YAML frontmatter block and a markdown body. The frontmatter declares metadata; the body is the instruction text that eventually becomes part of an agent's system prompt (or a tool description).

```
---
name: Redact
description: Mask identifiers before they reach the LLM
type: workflow
owner: [chat_agent, data_manager]
mode: inline
inputs:
  text: str
outputs:
  redacted: str
```

```
---

# Purpose
You are the Redact step. Given arbitrary text, return a redacted version...
```

**Frontmatter fields:**

| Field | Values | Meaning |
|---|---|---|
| `name` | string | Human-readable identifier |
| `description` | string | Used as the tool description when mode=tool |
| `type` | `workflow`, `domain`, `helper` | Folder the file lives under |
| `owner` | list of agent names | Which agents claim this skill |
| `mode` | `inline`, `tool` | How the body reaches the LLM (see below) |

Anything outside those five reserved keys goes into `meta` and is available for agent-specific logic (e.g. `inputs`, `outputs`, `tools`, `replaces`).

### The loader: `skills/loader.py`

`load_skill(path)` is the single entry point:

```python
# 1. Read the file
text = path.read_text(encoding="utf-8")

# 2. Split on the --- / --- fence
match = _FRONTMATTER_RE.match(text)   # regex: DOTALL, anchored at \A

# 3. Parse YAML frontmatter
fm = yaml.safe_load(match.group("frontmatter"))

# 4. Separate reserved keys from extras
common = {k: fm[k] for k in reserved if k in fm}
meta   = {k: v for k, v in fm.items() if k not in reserved}

# 5. Return a typed Skill object
return Skill(**common, body=match.group("body").strip(), meta=meta, path=path)
```

`load_skills_for(agent_name)` wraps this: it `rglob("*.md")`s the entire `skills/` tree, calls `load_skill` on each file, and keeps only those whose `owner` list contains the requested agent name. Parse failures are silently skipped so one broken file doesn't crash the agent.

### Two delivery modes

Once loaded, the skill body reaches the LLM via one of two paths, controlled by the `mode` field:

**`mode: inline` — injected into the system prompt**

`render_inline_prompt(skills)` concatenates all inline-mode bodies:

```
=== Redact ===
<body of redact.md>

=== Team Construction ===
<body of team_construction.md>
```

This string becomes part of the agent's `instructions=` argument. The LLM reads it at every turn as standing instructions.

**`mode: tool` — exposed as a callable tool**

`helper_tool_specs(skills)` returns a list of dicts (name, description, signature, body). In `agent_factories/helper_tools.py`, these are wrapped as callable Python functions whose `__doc__` is set to the skill body:

```python
def _with_doc(fn, skill_body):
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    wrapper.__doc__ = skill_body   # LLM reads this as the tool description
    return wrapper
```

The SDK's `bind_tools()` reads `__doc__` as the tool description, so the LLM sees the markdown prose as the explanation of what the tool does and when to call it.

### How agents consume skills in practice

Each agent factory loads specific skill files by direct path — not via `load_skills_for`. This makes the dependency explicit:

```python
# agent_factories/orchestrator_agent.py
_load_skill(_WORKFLOW_DIR / "team_construction.md").body
_load_skill(_WORKFLOW_DIR / "data_catalog.md").body
_load_skill(_WORKFLOW_DIR / "synthesis.md").body
_load_skill(_WORKFLOW_DIR / "balancing.md").body

# agent_factories/specialist_agent.py
_BASE_INSTRUCTIONS = _load_skill(_WORKFLOW_DIR / "data_query.md").body

# agent_factories/report_agent.py
_NEEDLE_PROMPT   = _load_skill(_WORKFLOW_DIR / "report_needle.md").body
_ANALYSIS_PROMPT = _load_skill(_WORKFLOW_DIR / "report_analysis.md").body
```

All of these `.body` strings are module-level constants — loaded once at import time, reused for every agent instantiation in that process.

### End-to-end picture

```
skills/workflow/team_construction.md
      |
      | load_skill()          <- parse frontmatter + body
      v
Skill.body  (plain string)
```

```
      |
      | _compose_orchestrator_instructions()
      v
"=== Team Construction ===\n<body>\n\n---\n\n=== Data Catalog ===\n..."
      |
      | Agent(instructions=...)
      v
System prompt  <-- LLM reads this on every turn
```

For helper tools the path diverges after `Skill.body`:

```
Skill.body
      |
      | _with_doc(acropedia_lookup, body)
      v
wrapper.__doc__ = body
      |
      | model.bind_tools([wrapper, ...])
      v
Tool description in the API request  <-- LLM reads this when deciding
                                          whether to call the tool
```

### Relationship to the redaction step

The `redact.md` skill is mode `inline` — its body is injected into the chat agent's and data manager's system prompts as standing instructions telling the *LLM* what patterns to mask when it generates output. `sanitize_message()` / `redact_payload()` in `firewall_stack.py` are the Python-side enforcement: they mask the same patterns on data *before the LLM ever sees it*. The two layers are complementary:

| Layer | What it does | When it runs |
|---|---|---|
| `redact.md` (inline skill) | Instructs the LLM to mask in its own output | At generation time |
| `sanitize_message` (Python) | Masks inbound text before it reaches the LLM | Before every API call |
| `redact_payload` (Python) | Masks structured tool results recursively | After every tool execution |

---

## 9. When Does the Specialist See the Tool Description?

The specialist is built with `data_query.md` as its system prompt and the `@function_tool` callables as its `tools=` list. Both reach the LLM in **every API request, simultaneously**.

### What gets sent in each request

```
API request
+-- messages[0]  (system)
```

```
|     +-- _compose_instructions()
|           +-- data_query.md body        <- tool guidance prose
|           +-- Domain: <skill.name>
|           +-- Data hints, risk signals, ...
|
+-- tools: [
      { name: "query_table",
        description: "<wrapper __doc__>", <- @function_tool schema
        parameters: { ... }              },
      { name: "aggregate_column", ... },
      ...
    ]
```

Both land in the same request. The specialist reads them simultaneously, before generating any response.

**They serve different purposes**

**`data_query.md` (system prompt) – strategic routing:**

> "When a question asks for shape over time, call `summarize_trend` once – never loop `aggregate_column` per period."

It tells the LLM *when* to call each tool, in what order, and why. This is the decision layer.

**`@function_tool` docstring (tools array) – tactical schema:**

> "For `filter_op='between'` pass `<low>,<high>` (inclusive)."

It tells the LLM *how* to form the arguments once it has already decided to call the tool. This is the invocation layer.

**The deliberate redundancy**

`data_query.md` names and describes each tool in prose. The `@function_tool` wrapper describes it again formally in the schema. The specialist sees the same tool twice – once as strategy, once as contract. That redundancy is intentional:

| Source | Layer | Purpose |
|---|---|---|
| `data_query.md` body in system prompt | Strategic | Routing rules, when to use each tool, ordering |
| `@function_tool` wrapper `__doc__` in tools array | Tactical | Argument shapes, parameter constraints |
| `_xxx_impl` docstring | Developer only | Internal logic, never sent to the API |

The `_impl` docstrings are the escape valve: all internal detail that neither the LLM nor the schema needs lives there, invisible to every API call and never billed as input tokens.

**Token cost**

Both the system prompt and the tools array are billed as input tokens on every turn. The specialist pays for `data_query.md`'s routing prose AND each wrapper's description on every single call – before a word of user question is counted.

---

## 10. Summary

| Concept | One line |
|---|---|
| Tool call | LLM *requests* your code to run something; you run it and report back |
| Tool schema | JSON description of a function's name, params, and purpose |
| `tool_call_id` | Ties each result to the specific request that triggered it |
| Redaction step | PII masking applied to every message before the firewall sees it |
| `sanitize_message` | Replaces 6+ digit runs and `CASE-\d+` tokens with masked placeholders |
| `redact_payload` | Recursively sanitizes structured tool outputs (dicts, Pydantic, lists) |
| SafeChain constraint | No native tool-calling; schemas injected as text; JSON response parsed back into SDK objects |
| `FirewallStack` | Shared state: logger, max_retries (2), concurrency semaphore (cap 3) |
| `FirewallRejection` | Typed exception carrying HTTP code + message; triggers retry-with-guidance |
| `_inject_guidance` | Re-redacts all messages and appends FIREWALL_GUIDANCE to the system message |
| Concurrency cap | Semaphore limits parallel in-flight requests to avoid token-rate exhaustion |
| Skill file | `.md` with YAML frontmatter (name, type, owner, mode) + instruction body |
| `load_skill` | Parses one `.md` into a typed `Skill` object (frontmatter + body) |
| `mode: inline` | Skill body concatenated into the agent's system prompt |
| `mode: tool` | Skill body attached as `__doc__` to a callable; LLM sees it as a tool description |
| `data_query.md` in system prompt | Strategic layer: routing rules for when and why to call each tool |
| `@function_tool` wrapper `__doc__` | Tactical layer: argument shapes and parameter constraints |
| `_impl` docstrings | Developer-only documentation; never sent to the API |